KurveRSC 0.2.0: Generic Staged Relational Signal Compression

Schema-Derived Representations, Temporal Selection, and Reproducible Prediction

Wes Madrigal
Kurve AI

November 5, 2026 | Maintainer-reproduced results

Contents

1 Abstract **2**

2 Scope, release identity, and revision **2**

3 Relational compression as a learned program **3**

4 Temporal and data contracts **4**

5 Representation space **5**

6 Staged graph search and increasing fidelity **6**

7 A symmetric production-faithful finalist check **6**

8 Temporal training policies and the generic optional menu **7**

9 Execution, portability, and production use **8**

10 Maintainer reproduction and release qualification **9**

11 Cost, resources, and parallelization **12**

12 Related work and architectural claims **13**

13 Limitations and next steps **14**

14 Appendix A: release and implementation provenance **15**

References **16**

1 Abstract

KurveRSC selects a relational feature program with a supervised learner in the loop. GraphReduce executes that program, reducing timestamped, related records to an entity-level feature matrix; CatBoost evaluates and fits the resulting representation. The staged procedure allocates increasing fidelity to a bounded set of schema-derived candidates, checks finalists with the production fitting lifecycle, and compares temporal support policies and generic optional readouts before saving keyed predictions.

This report specifies the published **KurveRSC 0.2.0 / GraphReduce 1.10.18** procedure.

The fixed profile is `kurversc-generic-staged-v1`. It replaces the October 27 report after removal of specialized recovery components. The retained optional menu contains parent text heads, training-frame ensembles, target-domain projection, a zero-aware distributional readout, and object breadth. All optional candidates challenge the same incumbent with one temporal admission rule. Their eligibility depends on declared inputs and capacity, independently of previous admissions. The finalist check now treats binary classification and regression symmetrically, using a 1% relative-loss cap on `1 - AUC` and MAE, respectively.

RelArena's maintainers reproduced all 21 RelBench v1 tasks at seed zero with these public packages and merged the system on November 5, 2026. On the frozen 16-method combined leaderboard, KurveRSC ranks **second overall and first among systems**, with **1913.4 Elo** (95% task-bootstrap interval 1832.3-2050.6), versus 1933.6 for the latest TabPFN-Rel client. It wins 11 of their 21 task comparisons. The broad overlapping intervals do not establish a statistically significant difference in aggregate performance. This report includes every reproduced per-task score and the successful jobs' measured resources [1], [2], [3]. Earlier author-run results remain a separate historical record.

The architecture separates relational representation from the downstream learner and from the execution backend. It supports operational use through the Kurve product, while the evaluated open-source recipe uses DuckDB. Warehouse execution and customer-controlled deployment are architectural and product capabilities; they do not establish an unlimited scale or runtime guarantee for this complete search procedure. The main unresolved questions remain selection fidelity, computational cost, and predictive transfer to previously unseen databases.

2 Scope, release identity, and revision

This document describes the fixed staged procedure rather than every native KurveRSC option. Its software dependencies are the public packages `kurversc[staged]==0.2.0` and `graphreduce==1.10.18`. The corresponding source identities are recorded in Appendix A for audit; installation uses package versions, not a Git dependency. The baseline relational-compression formulation is described in the original report [4].

The earlier October 27 report and its PDF are preserved as historical records. Their pipeline included additional, disputed recovery stages. Its scorecard cannot be relabeled as a measurement of the narrowed release. Strict temporal compliance of those earlier predictions was also not established. Neither source-level corrections nor transfer experiments retroactively establish their compliance.

2.1 What changed

Area	Published generic procedure
Core stages	Schema-derived graph search, full-fidelity rerank, production-faithful finalist check, temporal training-policy selection.
Generic optional menu	Text, frame ensembles, target-domain projection, zero-aware distributional readout, object breadth.
Finalist symmetry	The same bounded comparison for MAE and binary ROC-AUC; regression decisions preserved.
Optional admission	One selector, shared origins/seeds and incumbent, independent applicability, menu-size multiplicity adjustment.
Source boundaries	Explicit strict event cutoffs, separately declared availability, phase-censored task contexts and verified replay bindings.
Evidence	Maintainer reproduction of all 21 tasks, published combined leaderboard, per-task scores and measured job resources.

The runnable procedure removes exposure-role reconstruction, the recurrence/semantic teacher and offset-correction stack, conditional-count recovery, production-sequence recovery, bounded-view recovery, and benchmark test-win campaign gates. None is an optional fallback in the released staged execution closure. In particular, the procedure does not infer click, position, or prior-rate roles by alphabetic order or cardinality; it does not choose a rarest binary event column for a special head.

These removals do not remove ordinary GraphReduce `sequence`, `conditional`, `context`, or `state` features. Those are schema-derived feature families inside the graph search. Nor do they remove mature historical labels or generic peer context. A family of SQL summaries and a later specialized prediction-recovery stack are different components.

The fixed recipe retains globally specified model sizes, blend menus and capacity limits. They are shared defaults disclosed below, not values rediscovered on each database. Removing name-specific behavior does not prove predictive transfer; that remains an empirical question requiring new-database evaluation.

2.2 Interfaces

Interface	Responsibility
`kurversc.fit()` / `predict()`	Native graph search, fitting, and frozen-plan replay. The caller chooses settings.
`kurversc.run_staged_task()`	The complete fixed staged procedure described here.
RelArena registered system	Converts harness-owned splits into the task context, invokes the packaged procedure, and returns keyed predictions for external scoring.

Calling native `fit()` with its defaults does not by itself reproduce every stage of the submitted benchmark procedure. The retained RelArena system name is `kurversc-2026-10-22`; the date is a registry identifier, not a claim that the October 22 research recipe remains in use.

3 Relational compression as a learned program

Let a prediction request be (e, t), identifying a root entity and query cutoff. A relational database supplies keyed tables, foreign keys, source clocks, and observed labels. A graph configuration g selects paths, depth, reductions, feature families, and bounded annotation choices. GraphReduce computes

$$z_{e,t} = \Phi_g(D_{<t}, e, t), \qquad \hat{y}_{e,t} = f_\theta(z_{e,t}).$$

The notation $D_{<t}$ abbreviates the visibility contract in the next section; a source record may also have an independent availability clock. KurveRSC fits f_θ on eligible training examples and uses temporal validation to choose among programs. It therefore selects the compression procedure jointly with a bounded downstream fitting procedure, rather than creating a fixed flat table once and treating it as the entire database.

A many-to-one relationship contributes child summaries at the parent grain. Those features may continue upward through additional joins and reductions. Path-specific aliases distinguish repeated occurrences of a base table. Structural keys route records; they are excluded from the CatBoost payload, along with the current target and routing columns. Entity history can still contribute through explicitly constructed, time-safe relations.

The matrix is an interface to the learner. Information can be lost in the chosen representation, but a matrix interface does not imply that the features were computed from one original table. Conversely, adding columns cannot recover distinctions already erased by a reduction. Two entities with the same means and counts may have different event ordering, category-conditioned outcomes, or neighbor structure. The families below preserve some of these distinctions within a finite program budget.

The benchmark learner is CatBoost [5]. Other native learner integrations are separate interfaces; this release does not substitute a foundation model during the staged benchmark search or final prediction.

4 Temporal and data contracts

4.1 Source time and knowledge time

For a strict timed source row r, a request cutoff t, and a phase knowledge boundary b, source visibility requires

$$\tau(r) < t, \qquad \tau(r) < b.$$

When an independent availability clock $a(r)$ is declared, the phase export also requires $a(r) \leq b$. Event time and availability answer different questions: when an event occurred, and when the forecasting procedure could know it. Advancing a query clock must not advance a frozen phase's knowledge. RelBench timed-table adapters declare strict event boundaries. An explicit strict boundary cannot be relaxed by a legacy compatibility offset.

Filtering precedes annotation, category discovery, and feature materialization. The task context creates censored phase exports before provider discovery. Prediction contexts contain request keys and permitted source data, without held-out answers. Validation truth is used at the declared validation phase; held-out test scoring happens outside the prediction procedure after selection and keyed predictions are saved.

A source without clocks must be explicitly declared timeless. That declaration is a caller responsibility, not an inference that an undated mutable table was historically available. A source clock also does not reconstruct past versions of overwritten values. Correct point-in-time inputs remain necessary.

4.2 Historical target relations and peer context

Observed targets can provide useful prior state when the entity key is removed from the learner. KurveRSC can expose mature training labels as a bounded child relation. A label's query timestamp being earlier than the current request is necessary but insufficient: its prediction horizon must have matured, and any declared label availability must satisfy the relevant training boundary. Held-out targets are never added to this relation.

The history provider uses the observed-label source rather than the sampled CatBoost rows. Consequently, a model-row cap need not erase earlier known labels. The staged recipe also bounds peer-history discovery to 50,000 history rows. Counts, means, recency, and peer summaries are therefore bounded representations rather than a promise to expose every historical observation in every provider.

Generic peer context uses declared relationships and visible history. It can summarize other entities attached to common objects while excluding the current root's own contribution where the peer definition requires it. Peer-label availability may be stricter than the general source-export rule; the provider applies its own conservative visibility checks. This is separate from the removed exposure and bounded-view heads.

4.3 Request preservation and replay

A strict cutoff can hide the root row itself, for example when it was created exactly at the request time. The forecast request remains valid: its key and position are retained, and unavailable payload becomes missing. Dropping such requests would change the evaluation cohort.

A frozen plan records source bindings, path instances, operations, feature schema, category choices, and applicable numeric cutpoints. Prediction restores and verifies those bindings rather than rediscovering a graph. The source superset must support every offered repeated-table configuration. The current implementation corrects a mismatch between the offered path space and the source graph and checks restored bindings during replay.

5 Representation space

5.1 GraphReduce feature families

The fixed procedure builds candidates from applicable families and declared schema relationships. The following descriptions concern ordinary GraphReduce features, not the removed recovery heads [6].

Family	Signal represented
`base`	Standard numeric and categorical reductions, with bounded numeric distribution sketches in this recipe.
`temporal`	Numeric magnitude, dispersion and change within historical windows.
`state`	Latest values, ordered lags and summaries of the last several events, useful for irregular event cadence.
`sequence`	Activity rates, recent shares, concentration, bursts and active spans.
`conditional`	Counts, shares, presence and change under bounded categorical or annotated conditions.
`episode`	Event opportunity through row and distinct-event counts, including windowed counts.
`context`	Peer-group size and numeric deviations computed on child rows before reduction.

GraphReduce also accepts caller-declared semantic expressions and predicates. These are available natively; the fixed staged recipe does not automatically invent arbitrary semantic expressions or add the removed semantic correction stack. Parent/root payload and applicable annotations remain available, subject to the same source visibility and payload budgets.

5.2 CDF sketches inside `base`

For an eligible numeric payload v, bounded training-only quantile cutpoints c_j summarize more than a single mean. For entity e at cutoff t, the empirical fraction is

$$\widehat{F}_{e,t}(c_j) = \frac{\sum_{r \in R(e,t)} \mathbf{1}[v_r \text{ finite}]\mathbf{1}[v_r \le c_j]}{\sum_{r \in R(e,t)} \mathbf{1}[v_r \text{ finite}]}.$$

GraphReduce fits up to seven cutpoints from at most 20,000 eligible training rows and also records finite-value coverage. Empty support stays missing; constant, binary and entirely unobserved columns do not create these sketches. Cutpoints are frozen in the execution plan and are not refitted on validation or forecast data.

CDF is part of `base`, not an additional family. **The fixed staged recipe sets `base_cdf=True`.** Native and diagnostic interfaces can compare CDF on/off, but the published recipe does not double its entire graph menu into both forms. A bounded sketch preserves some distributional differences; it does not retain the complete empirical distribution or every cross-column dependency.

5.3 Depth, repeated tables, and payload limits

Candidates can reach depth four. An occurrence limit of one gives paths without repeated base tables; a limit of two admits bounded self/peer constructions. The executable graph remains a DAG because repeated tables have distinct path instances. This permits bounded walks through a cyclic relational schema without unbounded traversal.

The staged source-column budget is four payload columns per family/node, up to 32 derived features per selected source column, and one continuation function per propagated derived column. Structural keys, dates and context keys remain outside the payload budget. Deterministic column ranking uses coverage, entropy, spread, cardinality and identifier redundancy. The feature audit stores utility rank, selection rank, strategy and budget eligibility.

After promotion, categorical refinement tests bounded tie choices for up to two parent shapes. Available tie seeds are 0 through 7; the stage permits eight evaluations in total. These are distinct from the three model seeds used in procedure validation. Modern high-cardinality handling and a compatible anchor provide bounded alternatives; no payload policy exposes an unlimited set of identifiers or free-text features.

6 Staged graph search and increasing fidelity

Exhaustively fitting all depth, family, annotation and topology combinations at full fidelity is expensive. The `staged-retention` profile evaluates a bounded schema-derived menu, promotes survivors, then increases data fidelity. Search uses no saved task-to-winning-configuration lookup.

Stage	Max.	Purpose
Screening	32	Initial comparison, nominally 10,000-example frames.
Family/topology challengers	24	Applicable alternatives around retained candidates.
Combined challengers	12	Bounded combinations of promising choices.
Confirmation	40	Higher-fidelity comparisons, nominally 50,000-example frames.
Historical promotion	8	Additional eligible historical-cutoff evidence.
Categorical refinement	8	Deterministic categorical tie alternatives.
Full rerank	12	Frozen-plan replay on full eligible source frames and up to three recent cutoffs.

These ceilings sum to 136 stage evaluations. They are neither a count of unique graphs nor a count of CatBoost fits. Applicability, duplicate configurations, available support and stage outcomes can reduce evaluations. Multiple cutoffs, seeds, temporal policies and optional heads can increase learner fits beyond that sum.

Low-fidelity sampling can affect the connected source cohort as well as the examples scored by the learner. A 10K screening frame is therefore not always a cheap, otherwise identical estimate of the full-source graph. Rare categories and long relational paths can change. Frozen operations are replayed during promotion; full rerank removes the source-cohort sampling cap for its eligible frames. The downstream training-row limit remains separate.

The staged recipe caps graph features at 10,000 and uses a 100,000-row learner fit limit. The fixed CatBoost budget is 300 iterations, depth six, learning rate 0.05 and L2 regularization 10. These bounds control individual work items; they do not bound total materialized source data or total process-tree RAM.

Search support is distinct from deployment support. The initial production policy remains `full_training_frames=1`, even when search or rerank evaluates several historical cutoffs. Wider production support must win as a complete training-policy candidate. Widening validation must not silently turn the final model into an equal-weight average of old models.

The native graph selector uses recent-cutoff performance, older-cutoff stability, uncertainty and a bounded complexity preference. This is a heuristic for choosing among observed candidates. It cannot recover a graph discarded before full fidelity, nor guarantee the globally best program. The additional finalist comparison below specifically checks the production fitting lifecycle.

7 A symmetric production-faithful finalist check

After full rerank, S2 considers at most one smaller completed finalist. It must have a frozen plan and complete evidence on at least two cutoffs. Screening compares recent validation loss with the best completed recent loss; the candidate must be within 1%. The bounded check proceeds only for an alternate with at most half the incumbent's feature count and sufficient evidence and capacity. Otherwise it retains the incumbent.

Binary ranking and numeric regression use a common lower-is-better scale:

$$L(s) = \begin{cases} 1 - s, & s = \text{ROC-AUC}, \\ s, & s = \text{MAE}\,. \end{cases}$$

Here `1 - AUC` is pairwise ranking loss, counting a tied positive/negative pair as half an error. It is not classification error. At AUC 0.80, a 1% relative loss tolerance is 0.002 AUC; at AUC 0.98 it is 0.0002 AUC. The same rule therefore has comparable sensitivity on a relative loss scale rather than allowing a fixed 0.01 AUC loss near perfect ranking.

Incumbent and candidate are compared on matching origins and seeds 0, 1 and 2, using the native initial-fit/observed-validation-refit lifecycle. The native metric, uncertainty estimate and graph selector are applied independently for each seed. Replacing the incumbent requires unanimous challenger selection and, for every compared seed and fold,

$$L_{\text{candidate}} \leq 1.01 L_{\text{incumbent}} + 10^{-12}.$$

Incomplete comparisons do not promote a partially evaluated alternate. Capacity preflights and a nested stage timeout bound this extra work. This comparison is not another unrestricted graph search and does not change the downstream learner budget.

Earlier versions applied S2 only to MAE. Version 0.2.0 extends it to both supported metrics using the same comparison mechanics. Thirty regression before/after comparisons reproduce the prior MAE decisions and evidence exactly. Classification has explicit acceptance and veto coverage. Neither check establishes that the smaller graph will generalize better in every case.

8 Temporal training policies and the generic optional menu

8.1 Training support and deployment lifecycle

S3 evaluates complete temporal fitting policies against the selected graph on up to three eligible origins and seeds 0, 1 and 2. The initial incumbent uses the most recent complete training cutoff, with the declared observed-validation refit lifecycle. Alternatives include bounded pooled support, ordered regression readouts and temporal blends. The menu can consider up to twelve support cutoffs with a row floor of 1,000, subject to maturity and available history.

Pooled training applies a shared model-row budget; frame ensembles can fit that budget per member. These have different computational and statistical meanings. Three validation seeds test repeatability of selection; they are not a default three-seed inference ensemble. Deployment uses seed zero. Nor does selecting three graph finalists imply that prediction averages three graphs: one graph is selected before temporal policy and optional readout selection.

8.2 One optional menu and one incumbent

Each optional candidate challenges the same S3 incumbent, on the same phase exports and validation cases. Eligibility follows required inputs and declared capacity, independently of whether another optional candidate wins. There is no recovery cascade activated only when a preceding head keeps the incumbent.

Candidate group	Applicable inputs and globally shared action
Parent text	Eligible root categorical/text payload; bounded TF-IDF plus logistic or ridge readout, blended at 0.25 or 0.5.
Frame ensembles	Mature historical support; complete three- or five-frame fitting procedure.
Target domain	Regression targets observed to be nonnegative or integer counts; clip negative predictions or apply the integer-domain projection.
Zero-aware	Nonnegative regression with adequate zero and positive examples; occurrence model plus positive-amount quantile readout, blended at 0.125, 0.5 or 1.
Object breadth	Declared timed root-to-object relationships; object-history features with a fitted learner, full prediction or a 0.5 blend.

There are eleven named candidate recipes before applicability filtering. The menu is fixed globally, not chosen from a table of benchmark outcomes. The following settings define bounded implementations, not an assertion that these values are optimal for every database.

The text head uses one shared text definition: at least 20 distinct values and mean nonempty length at least 32 characters. It uses at most eight root fields, truncates each to 2,048 characters, fits a training-only TF-IDF vocabulary of at most 16,384 word unigrams/bigrams, and samples at most 100,000 training examples. Classification uses logistic regression with $C = 1$; regression uses ridge regression with $\alpha = 10$. This lexical head does not require MiniLM, PyTorch or Transformers. Character length is an imperfect proxy for language; admitting a column does not prove it contains semantic text.

Target-domain projection uses eligible observed training targets, not test targets. The integer projection is distinct from the removed Poisson conditional-count head. The zero-aware candidate requires at least 20 zero and 20 positive examples and fits positive-amount quantiles at $\{0.05, 0.25, 0.5, 0.75, 0.95\}$. Its readout approximates a predictive mixture with a mass at zero [7].

Object breadth considers timed declared paths to keyed objects with at least 64 visible distinct objects, retaining the top two paths by distinct-object count. Its summaries cover object/event counts, new and singleton objects, maximum share and concentration over horizon-scaled windows. Source lookback is capped at 730 days. At most two paths produce 42 features. The cap can truncate a long-horizon window; using more eligible paths and scaling history limits with the task are remaining generality improvements, not changes in this release. The frozen aggregate query runs serially from the first query to avoid an unsafe native DuckDB parallel-combine path without changing the query, rows or features.

8.3 Shared temporal admission rule

The optional selector requires at least two origins and three seeds. Every latest-origin seed must improve over the incumbent. Older origins supply a practical safety veto: no more than 0.005 AUC or 1% relative MAE degradation under the selector's older-origin checks. A paired, entity-cluster bootstrap uses 600 draws with $\alpha = 0.05$ and a menu-size multiplicity adjustment; the temporal gain lower bound must be positive [8]. If adequate support is unavailable, the incumbent stays selected.

The recent-only lower bound is recorded as a diagnostic and is **not** a separate positive-bound requirement. This definition is shared by optional heads; there is no stricter, differently tuned gate for a selected recovery stage. S2's graph-complexity comparison is a separate operation with the explicit relative-loss rule above, not this optional admission test.

These checks reduce unsupported promotions. They do not guarantee no future regression, account for every hypothesis examined during development, or prove transfer to an unrelated database. A global blend menu is still a finite search space; adequate fresh evaluation remains necessary.

9 Execution, portability, and production use

9.1 Explicit task context

A task supplies source tables and keys, relationships, event/availability clocks, a root entity, a label horizon, observed labels, and masked forecast requests. `TaskContext.from_tables` supports caller-owned unregistered schemas; `TaskContext.from_protocol_splits` supports already censored protocol phases. Neither requires a RelBench dataset name or a saved winning graph.

```
python -m pip install "kurversc[staged]==0.2.0" "graphreduce==1.10.18"
```

```
import kurversc

# context declares the schema, observed labels, clocks, and masked requests.
result = kurversc.run_staged_task(
    context,
    output="artifacts/task",
    task_hours=23.75,
    minimum_available_memory_gib=8,
)
predictions = result.predictions
```

Prediction artifacts preserve entity/cutoff keys and ordering. Manifests record context/source identities, selected graph and execution plan, temporal policy, optional admission and prediction fingerprints. Resume uses matching completed artifacts and retains the original absolute task deadline; it does not grant a new day for each restart. Incomplete resource-stopped tasks are not reported as successful scored tasks.

9.2 Benchmark boundary

The RelArena adapter invokes the packaged staged entry point with an explicit portable context. The harness owns outer held-out scoring. A standalone research campaign that accesses a raw RelBench cache is a different execution path and cannot establish that a harness reproduction occurred. The reported release qualification uses the registered adapter and actual core split types; the completed reproduction identifies the adapter, public packages, source boundaries and final result rows [9], [10].

The package's transitive staged execution closure includes 32 stage modules. It excludes the removed recovery stages. Historical scripts remain in source history for provenance, but are not implicit runtime choices. The staged extra no longer requests the removed semantic encoder's deep-learning dependencies.

9.3 Warehouse backends and the Kurve product

GraphReduce separates relational operations from execution, with backend support that includes production SQL environments such as Snowflake, Databricks and Redshift. SQL compilation and warehouse execution can place relational feature computation near governed source data. The frozen benchmark path uses DuckDB [6], [11]. It does not demonstrate that every staged Python-side materialization and fit runs entirely inside every warehouse, or that any SQL engine is interchangeable without backend qualification.

The Kurve product provides schema/metadata graphs, parent-node and compute-graph workflows, execution and model workflows, and integration with existing ML/MLOps stacks. Customer-controlled deployment can keep data and model operation in the customer's environment, supporting sovereign-AI requirements [12], [13]. These are production delivery options; sovereignty also depends on deployment, access controls and external-service configuration. They are distinct from benchmark rank and from an unrestricted scale claim for this search implementation.

10 Maintainer reproduction and release qualification

10.1 Release qualification (November 4, 2026)

Check	Evidence and scope
Source suite	727 passed, one skipped, on the release source.
Adapter/discovery suite	19 passed against the current registered RelArena interface.
Finalist regression parity	Thirty matched comparisons preserve prior MAE decisions and evidence.
Classification finalist checks	Explicit positive/veto cases for ranking loss, uncertainty, origins and seeds.
Installed classification run	Complete registered procedure on a fresh, unregistered 48-request synthetic schema; no hidden targets.
Installed regression run	The same execution qualification for numeric regression on a different fresh schema.
Package parity	Source and installed public-wheel Python payloads match: 80 KurveRSC and 22 GraphReduce modules.
Dependency lock	The submitted 215-package lock resolved both exact versions from PyPI.

The two synthetic runs completed in approximately 161 and 156 seconds. Their small schemas do not exercise every possible head or guarantee an S2 alternate is eligible. They establish end-to-end execution, not meaningful predictive quality or a large-database resource envelope. Qualification reused existing numerical dependencies; it is not a cold installation of every dependency on every supported platform. Detailed source checks are recorded in the qualification receipt.

10.2 Reproduced evaluation and provenance

RelArena PR #80 was merged on November 5, 2026. The maintainers ran `kurversc-2026-10-22` through its registered adapter using public KurveRSC 0.2.0 and GraphReduce 1.10.18, at seed zero, one task per CPU host. The 21 selected rows in `baseline_results/results.csv` cover seven databases, twelve binary classification tasks and nine regression tasks. All final rows contain finite test scores and no error. These are the reference measurements for this report [1].

The snapshot is RelArena `fcab8fb` (November 5, 2026), after the staged-system merge and dating of the earlier KurveRSC registration. The immutable result file, published leaderboard and resource record identify the exact comparison. The machine-readable extract retains full-precision scores, timings, source checksums and model identifiers.

10.3 Combined leaderboard

The method column credits the underlying model, architecture, pretraining-data work or official release, as applicable. RT-PluRel is credited to the RT and PluRel papers; RT-J is credited to those foundations and the RT-J paper. The constant predictors use RelArena's documented baselines. These method citations identify intellectual provenance; every displayed score and interval still comes from the frozen RelArena reproduction, rather than from the cited papers.

The published **models plus systems** panel contains sixteen complete methods on the same 21 tasks. It excludes self-reported `_MR` reference rows and incomplete/exploratory runs. Ratings are anchored to the global constant at 1000. Intervals use the 2.5th and 97.5th percentiles of 100 task-bootstrap samples; they describe uncertainty across tasks, not variation across training seeds. A models-only board is computed separately and excludes KurveRSC as a system; its ratings must not be substituted into this combined comparison [2].

Rank	Method	Kind	Elo	95% task-bootstrap interval
1	tabpfn-rel-client-latest [10], [14]	Model	1933.6	1864.7-2033.5
2	kurversc-latest (this work)	System	1913.4	1832.3-2050.6
3	tabpfn-rel-local-latest [10], [14]	Model	1887.7	1831.3-1974.8
4	rt-j [15], [16], [17]	System	1842.2	1721.9-1997.8
5	tabpfn-rel-client-2026-08-15 [10], [18]	Model	1799.0	1740.7-1886.9
6	rt-plurel [15], [16]	System	1787.7	1695.9-1889.4
7	kurversc-2026-09-03 [4]	System	1773.8	1709.9-1849.6
8	nori-rel [19], [20]	Model	1732.3	1660.5-1844.7
9	tabpfn-rel-local-2026-08-15 [10], [18]	Model	1702.0	1613.7-1771.7
10	graphsage [21], [22]	Model	1660.3	1550.8-1747.2
11	relgt [23]	Model	1562.7	1430.3-1664.5
12	rdblearn [24]	Model	1529.8	1437.8-1610.5
13	relgnn-es [25]	Model	1525.0	1421.2-1636.6
14	lightgbm [26]	Model	1363.9	1168.9-1501.1
15	constant-per-entity [10]	Model	1266.5	1082.8-1408.5
16	constant-global [10]	Model	1000.0	838.5-1076.9

KurveRSC's point rating is 20.2 below the latest TabPFN-Rel client and 139.6 above the earlier KurveRSC registration in this same panel. The intervals overlap substantially. A higher point rank is not proof of a significant pairwise advantage, and task-level wins are not interchangeable with Elo: Elo accounts for results against the full panel.

10.4 Classification: ROC-AUC, higher is better

The earlier column is `kurversc-2026-09-03`; the current column is `kurversc-2026-10-22`; the client column is `tabpfn-rel-client-2026-09-28`, which the snapshot displays as the latest client. All values below are held-out test scores from selected rows in the same published file. Winners are determined from full precision, before formatting [1].

Dataset / task	Earlier KurveRSC	KurveRSC 0.2.0	TabPFN-Rel client	Winner
Amazon / item-churn	0.827256	0.828712	0.828653	KurveRSC
Amazon / user-churn	0.710347	0.712558	0.711698	KurveRSC
Avito / user-clicks	0.658406	0.660062	0.691631	Client
Avito / user-visits	0.674721	0.672515	0.672453	KurveRSC
Event / user-ignore	0.832628	0.850888	0.784037	KurveRSC
Event / user-repeat	0.779679	0.782152	0.760495	KurveRSC
F1 / driver-dnf	0.746484	0.722017	0.727809	Client
F1 / driver-top3	0.664957	0.834154	0.791702	KurveRSC
H&M / user-churn	0.696916	0.707174	0.706021	KurveRSC
Stack / user-badge	0.877626	0.876020	0.883443	Client
Stack / user-engagement	0.903027	0.903861	0.907662	Client
Trial / study-outcome	0.712131	0.760476	0.764556	Client

10.5 Regression: MAE, lower is better

MAE is in each task's native units. These values are not averaged across tasks with different target scales.

Dataset / task	Earlier KurveRSC	KurveRSC 0.2.0	TabPFN-Rel client	Winner
Amazon / item-ltv	42.2731	42.3828	45.2705	KurveRSC
Amazon / user-ltv	14.0205	13.9894	13.8771	Client
Avito / ad-ctr	0.033440	0.033258	0.031258	Client
Event / user-attendance	0.258668	0.234934	0.241652	KurveRSC
F1 / driver-position	3.8632	4.0605	3.7574	Client
H&M / item-sales	0.032168	0.033636	0.036479	KurveRSC
Stack / post-votes	0.063243	0.062864	0.062733	Client
Trial / site-success	0.403842	0.398749	0.377615	Client
Trial / study-adverse	41.2702	33.1750	34.4014	KurveRSC

KurveRSC wins **11/21** observed task comparisons against the client: seven classification and four regression tasks. The client wins the other ten. Several margins, especially item-churn and user-visits, are small point advantages; a single training seed does not establish their robustness. Relative to the earlier KurveRSC registration, fifteen scores improve and six regress. The table discloses both directions; it is not a controlled ablation of any single feature or selection stage.

10.6 Relationship to earlier evidence

The six-task narrowed pilot predates source-replay and symmetric-finalist revisions and is no longer the current performance reference. An earlier Olist experiment used a superseded procedure; retaining its incumbent does not show positive transfer of every optional component [27]. The completed RelArena evaluation establishes benchmark performance of the revised release, while predictive transfer to new databases and per-component attribution remain separate questions.

The archived October 27 report records a different author-run source and comparison panel. Its Elo, temporal semantics and resources must remain tied to that source. No historical score is carried forward into the reproduced 21-task column above.

11 Cost, resources, and parallelization

11.1 Measured resources in the maintainer reproduction

The public resource record reports successful jobs on two CPU host classes: eight tasks on 96-vCPU, 739-GB hosts, and thirteen on 48-core, 190-GB hosts. The larger hosts used a 160-GiB DuckDB working-memory limit and a 100-GiB temporary-directory limit; the smaller hosts used 48 GiB and 20 GiB, respectively. One task ran per host. Host RAM is reported in GB; observed Slurm `MaxRSS` is reported in GiB [3].

Task	Host RAM, GB	Job wall time, h	Peak RSS, GiB
Amazon / item-churn	739	3.32	364
Amazon / item-ltv	739	6.48	202
Amazon / user-churn	739	4.98	262
Amazon / user-ltv	739	8.86	327
Avito / ad-ctr	190	0.71	34
Avito / user-clicks	190	1.21	51
Avito / user-visits	190	1.14	64
Event / user-attendance	190	3.35	46
Event / user-ignore	739	1.83	283
Event / user-repeat	190	1.72	44
F1 / driver-dnf	190	5.88	6
F1 / driver-position	190	4.88	4
F1 / driver-top3	190	3.93	4
H&M / item-sales	190	3.47	27
H&M / user-churn	190	1.64	21
Stack / post-votes	739	10.60	257
Stack / user-badge	739	10.38	370
Stack / user-engagement	739	4.36	219
Trial / site-success	190	1.73	89
Trial / study-adverse	190	6.36	30
Trial / study-outcome	190	2.27	23

The successful jobs take **0.71-10.60 hours** and report **4-370 GiB MaxRSS**. The separate application `time_total` values in the score CSV span 0.70-10.59 hours and sum to 88.81 task-hours. Job wall time and application time are different recorded clocks. Their sum is not the elapsed duration of a parallel multi-host campaign and does not include failed earlier attempts. Every successful reproduced task finishes below 24 hours.

Resource retries are part of the provenance: user-ignore and user-engagement ran out of memory on the 190-GB hosts with the 48-GiB DuckDB limit; post-votes reached 140 GiB there before being moved. All three completed on the 739-GB hosts. Consequently, this is a complete set of final results, **not a claim of 21 successful first attempts**. Peaks depend on engine limits and execution conditions and are observations, not minimum-memory requirements. Slurm MaxRSS is not interchangeable with the earlier author's process-tree RSS or cgroup memory peaks [3].

11.2 Execution budgets and parallel admission

The complete task budget is 23.75 hours, below RelArena's 24-hour limit. This is a deadline, not a guarantee that every schema can finish under it. Source size, join fan-out, emitted columns, historical cutoffs and optional learners all contribute to cost. A timeout or resource stop must remain visible in the results; it must not silently shorten the procedure and be called a complete reproduction.

Training-row and feature limits do not cap total RAM. SQL joins, annotations, frozen-frame caches, Python copies, CatBoost workspaces and concurrent graph workers may coexist. DuckDB working-memory settings govern spill thresholds, not the entire process tree. Several simultaneously active tasks can therefore require much more memory than the largest model matrix.

Current execution reuses matching plans and materializations rather than restarting a search after each later comparison. It separates source replay from model fitting where possible and avoids rediscovery during prediction. Such

reuse must preserve phase, source identity, cutoff, operations and schema; a cache hit with mismatched bindings is a correctness error.

Task-level and graph-configuration concurrency are distinct. Increasing both multiplies overlapping materialization and learner memory. Parallel execution should be admitted from measured process-tree or cgroup demand with an explicit host reserve. The maintainer measurements above provide current planning evidence, but concurrent tasks need additional headroom and may change spill behavior. Historical removed-stage peaks are a separate measurement series.

The published resource table identifies host classes, job wall time, Slurm MaxRSS and engine limits. It does not report every temporary-storage peak or CPU-utilization trace. A hardware-normalized speedup over previous author runs cannot be inferred from these different hosts and accounting methods. Further efficiency work should retain source/phase identity and disclose retries, concurrency and the resource measure used.

12 Related work and architectural claims

12.1 Relational feature construction and learned neighborhood representations

Deep Feature Synthesis and OneBM automate relational feature construction [28], [29]. KurveRSC follows the broad idea of constructing entity-grain features from related records, with supervised program selection, explicit temporal phases and frozen execution as central requirements.

RelBench and its larger v2 repository supply relational prediction tasks [22], [30]. GraphSAGE, relational deep learning, RelGNN and RelGT instead learn representations through neighborhood/message-passing mechanisms [21], [23], [25], [31]. These can retain or learn interactions that a finite set of independent reductions erases. KurveRSC does not claim representational equivalence to those models.

Deep Sets and work on graph-neural-network expressivity motivate careful choice of aggregation functions [32], [33]. They do not prove that this bounded, hand-specified feature library is universally sufficient. Equal compressed vectors are indistinguishable to any downstream learner; a stronger tabular model cannot reconstruct information absent from those vectors.

The Relational Transformer (RT) tokenizes database cells with schema metadata and attends across relational structure [15]. PluRel supplies synthetic relational databases for pretraining [16]. RT-PluRel combines that architecture and synthetic-data pretraining. RT-J builds on these foundations with continued pretraining on real relational databases from THE JOIN and context-efficient retrieval and inference [17]. In the RelArena panel, RT-PluRel and RT-J are fine-tuned systems with custom tuning; their reproduced scores should not be read as zero-shot results from the architecture papers [10].

TabPFN and subsequent tabular foundation models use pretrained inference [14], [18], [34]. TabPFN-Rel couples this learner family with relational feature construction, while RDBLearn provides an aggregation-to-tabular-inference framework [10], [24]. Nori-Rel pairs relational feature construction with the Nori foundation model; its official release description and the authors' requested software citation provide the corresponding credit [19], [20]. The entity-only LightGBM baseline uses gradient-boosted decision trees [26], while KurveRSC's benchmark learner is CatBoost [5]. KumoRFM explores a separate relational in-context architecture [35].

The TabPFN-3.5 technical report was most recently revised November 3, 2026 at this writing [14]. The benchmark comparators are the dated API/local submissions identified in the reproduction section; their reproduced scores are distinct from paper-reported results. Model papers, training-data papers and software releases are cited for their respective contributions, rather than treated as interchangeable methods.

12.2 What this approach says about flattening

KumoRFM-2 argues that task-agnostic aggregation can obscure useful relational structure, and motivates early task conditioning in relational prediction [36]. NVIDIA Kumo's explanatory documentation likewise emphasizes losses of topology, temporal structure and multi-hop interactions when constructing flat features [37]. These are substantive representation concerns. They are not a theorem that every learner consuming a matrix must be uncompetitive.

KurveRSC makes a different design choice: select a task-conditioned relational program before fitting the tabular learner. This addresses the gap between unselected summaries and supervised relational compression. It leaves open whether the available programs preserve the interactions a particular task needs. The maintainer-reproduced second-place overall result provides empirical evidence that this matrix-based learner interface can be competitive

after task-conditioned relational compression. It does not show that arbitrary flattening is lossless or establish a matched result against KumoRFM-2. That comparison would require the same tasks, inputs, temporal visibility, external pretraining disclosure and compute protocol.

12.3 Search allocation and uncertainty

Successive halving, Hyperband and BOHB study allocation of expensive evaluations [38], [39], [40]. The present stages use a related increasing-fidelity idea, but do not inherit a guarantee that 10K or 50K graph scores preserve full-source rankings. Their usefulness must be measured for this representation space and deployment lifecycle.

Paired temporal comparisons and cluster resampling account for some correlated entity observations. They do not turn reused development tasks into untouched tests. Conservative admission is a selection mechanism, not proof that a specialized transform generalizes. Future claims should emphasize fresh-schema evaluations and controlled ablations alongside aggregate benchmark scores.

13 Limitations and next steps

The most direct failure modes are representation loss, selection regret and cost. Bounded CDFs, category annotations, event windows and peer summaries do not retain every joint event or neighbor distinction. A frozen plan discovered on capped source cohorts can miss rare categories and fine temporal interactions. A stronger graph can exist in the space yet be screened out before comparable full-fidelity evidence is collected.

The next search evaluation should measure rank correlation, winner retention and selection regret across 10K, 50K and full-source fidelity on identical candidate sets. Finalist comparisons should match deployment support and refit behavior. Reporting only the final winning score does not reveal whether an improvement came from richer signal or a better selection decision.

Generalization also depends on source metadata quality, historical data availability, eligibility limits and validation support. Top-two object paths, a fixed 730-day object-history cap, and heuristic text detection are disclosed restrictions. Enumerating more valid alternatives may improve generality, but can increase search cost and multiple-comparison burden. Changes should be qualified with the same public procedure on held-out databases, without benchmark-name switches or test-score campaign gates.

13.1 Amortizing graph selection with a pretrained planner

A future planner could propose graph configurations from a schema, relationship metadata, table/column statistics, root, horizon and target type. It could warm start a bounded search while retaining verified execution and validation. Zero-shot prediction of the universally optimal graph is a stronger claim and has not been demonstrated.

The limiting resource is a training corpus of phase-correct tasks, evaluated programs, fidelity/cost records, uncertainty and independent held-out outcomes. Schema metadata alone does not label predictive quality. RelBench v2, 4DBInfer and the CTU repository can seed collection [30], [41], [42], but sibling tasks do not provide independent databases. THE JOIN, released with RT-J, offers another source of real relational databases and forecasting tasks [17]. A graph-selection planner would still need its own phase-correct program-quality and cost traces on those tasks. Permissioned operational traces and synthetic tasks could broaden coverage with appropriate licensing, privacy and temporal validity.

Evaluation must hold out entire databases/schema families. It should measure full-fidelity winner retention, regret, execution validity and cost against the fixed staged search. No pretrained planner is part of this release.

14 Appendix A: release and implementation provenance

The public dependency recipe is:

```
kurversc[staged]==0.2.0
graphreduce==1.10.18
```

Both were published on November 4, 2026. For source audit, the release snapshots are KurveRSC `cf6599bd7de21f32b299c3cdce75f611ad050a06` and GraphReduce `21aff2e9ad83d69d7fc468d01e3d56a0dc6ca976`. The latter is the code published as GraphReduce 1.10.18, not an unpublished local patch. Documentation revisions do not alter these released Python payloads.

The public artifact checksums are recorded below. They identify release files, not additional installation constraints.

```
kurversc-0.2.0-py3-none-any.whl SHA-256
0d5eae0f9381d0948475352bcae97b85e5fc5dce98ea4cfb1f7d50bceb635ea0

kurversc-0.2.0.tar.gz SHA-256
6093ba24d9d4fb31252980d816032c1141dab6ea9c50713e797428d458f5fc20

graphreduce-1.10.18-py3-none-any.whl SHA-256
5d99b358e4ce5f85c3d16fb8a2db072d9c18da6abbd03c4e194aff822410bd81

graphreduce-1.10.18.tar.gz SHA-256
20e7ac6335c6231c836e83339d69c98c895f30800fb83d5f7484fbb57960afc1
```

Implementation area	Source entry points
Public staged execution	`kurversc/staged.py`, `scripts/run_portable_staged.py`
Fixed generic contract	`scripts/run_admissible_staged.py`
Task/source boundaries	`kurversc/task_context.py`, `kurversc/task_source.py`
Search and frozen plans	`kurversc/core.py`, `scripts/run_kurversc_selected_relarena.py`
Finalist symmetry	`scripts/generic_finalist_helpers.py`, `scripts/benchmark_production_finalists_v2.py`
Temporal selection	`kurversc/temporal_policy.py`
Shared optional menu	`scripts/benchmark_generic_readouts.py`, `kurversc/signal_challengers.py`
Object-history boundary	`scripts/phase_frozen_object_breadth.py`
Packaged stage closure	`staged_build.py`, `kurversc/staged_runtime.py`

The current release specification, review/change record and qualification receipt provide companion evidence. Older result bundles remain tied to their recorded source identities; repository history is not an implicit source of runtime winners.

References

References are numbered in order of first citation. The bibliography covers the implementation, protocol, relational feature construction, learned representations, search allocation, statistical methods and proposed planner-data sources. Compact source labels remain linked to the complete original URLs. Product/deployment references describe capabilities rather than benchmark results. Software and revised online sources were checked on November 5, 2026; unchanged research-paper metadata is retained from the earlier report.

[1] Prior Labs, "RelArena: Maintainer-Reproduced KurveRSC 0.2.0 Results." 2026. Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[2] Prior Labs, "RelArena: Models and Systems Leaderboard, November 5, 2026." 2026. Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[3] Prior Labs, "RelArena: KurveRSC Maintainer Reproduction Resources." 2026. Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[4] W. Madrigal, "KurveRSC: Validation-Guided Relational Signal Compression with a Downstream Learner in the Loop," Kurve AI, 2026. Accessed: Oct. 26, 2026. [Online]. Available: GitHub source

[5] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased Boosting with Categorical Features," in *Advances in neural information processing systems*, 2018. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1706.09516

[6] W. Madrigal, "GraphReduce." Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[7] R. Koenker and G. Bassett Jr., "Regression Quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978, doi: 10.2307/1913643.

[8] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.

[9] Prior Labs, "RelArena: Open and Reproducible Benchmarking for Relational Learning." Accessed: Nov. 05, 2026. [Online]. Available: GitHub source

[10] A. Hayler, K. Floge, A. Arazi, R. Ranjan, *et al.*, "Advancing Open and Reproducible Relational Learning: RelArena-alpha, TabPFN-Rel and RPI." 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2608.16319v2

[11] M. Raasveldt and H. Muhleisen, "DuckDB: An Embeddable Analytical Database," in *Proceedings of the 2019 international conference on management of data*, 2019. doi: 10.1145/3299869.3320212.

[12] Kurve AI, "Kurve Product and Deployment." Accessed: Nov. 05, 2026. [Online]. Available: Kurve AI

[13] Kurve AI, "Kurve Product Documentation." Accessed: Nov. 05, 2026. [Online]. Available: Kurve documentation

[14] B. Jäger, N. Erickson, L. Grinsztajn, F. Birkel, *et al.*, "TabPFN-3.5: Technical Report." 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2609.17895

[15] R. Ranjan *et al.*, "Relational transformer: Toward zero-shot foundation models for relational data," in *The fourteenth international conference on learning representations*, 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2510.06377

[16] V. Kothapalli *et al.*, "PluRel: Synthetic data unlocks scaling laws for relational foundation models," in *Forty-third international conference on machine learning*, 2026. Accessed: Nov. 05, 2026. [Online]. Available: Publisher

[17] R. Ranjan *et al.*, "RT-J: Large-scale pretraining of relational transformers for context-efficient predictions," in *The fortieth annual conference on neural information processing systems*, 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2610.17867

[18] L. Grinsztajn, K. Floge, O. Key, F. Birkel, *et al.*, "TabPFN-3: Technical Report." 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2605.13986v2

[19] Synthefy *et al.*, *Synthefy Nori: Tabular Foundation Model for Regression.* (Jun. 2026). Zenodo. doi: 10.5281/zenodo.20710462.

[20] Synthefy Team, "Introducing Nori-Rel: Predictions for Your Relational Databases." Accessed: Nov. 05, 2026. [Online]. Available: www.synthefy.com

[21] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in neural information processing systems*, 2017. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1706.02216

[22] J. Robinson *et al.*, "RelBench: A Benchmark for Deep Learning on Relational Databases," in *Advances in neural information processing systems, datasets and benchmarks*, 2024. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2407.20060

[23] V. P. Dwivedi *et al.*, "Relational Graph Transformer," in *International conference on learning representations*, 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2505.10960v2

[24] Y. Zhang, L. Xu, Q. Gan, D. Wipf, and M. Wang, "RDBLearn: Simple In-Context Prediction Over Relational Databases." 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2602.18495

[25] T. Chen, C. Kanatsoulis, and J. Leskovec, "RelGNN: Composite Message Passing for Relational Deep Learning," in *Proceedings of the international conference on machine learning*, 2025. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2502.06784

[26] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in neural information processing systems*, 2017. Accessed: Nov. 05, 2026. [Online]. Available: Publisher

[27] Prior Labs, "Relational Predictions with TabPFN-Rel." Accessed: Oct. 26, 2026. [Online]. Available: GitHub source

[28] J. M. Kanter and K. Veeramachaneni, "Deep Feature Synthesis: Towards Automating Data Science Endeavors," in *IEEE international conference on data science and advanced analytics*, 2015. doi: 10.1109/DSAA.2015.7344858.

[29] H. T. Lam, J.-M. Thiebaut, M. Sinn, B. Chen, T. Mai, and O. Alkan, "One Button Machine for Automating Feature Engineering in Relational Databases." 2017. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1706.00327

[30] J. Gu *et al.*, "RelBench v2: A Large-Scale Benchmark and Repository for Relational Data." 2026. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2602.12606v2

[31] M. Fey *et al.*, "Position: Relational Deep Learning - Graph Representation Learning on Relational Databases," in *Proceedings of the 41st international conference on machine learning*, 2024, pp. 13592–13607. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[32] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. J. Smola, "Deep Sets," in *Advances in neural information processing systems*, 2017. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[33] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *International conference on learning representations*, 2019. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1810.00826

[34] N. Hollmann *et al.*, "Accurate Predictions on Small Data with a Tabular Foundation Model," *Nature*, vol. 637, pp. 319–326, 2025, doi: 10.1038/s41586-024-08328-6.

[35] M. Fey, V. Kocijan, F. Lopez, J. E. Lenssen, and J. Leskovec, "KumoRFM: A Foundation Model for In-Context Learning on Relational Data," Kumo.AI, 2025. Accessed: Oct. 26, 2026. [Online]. Available: Archived report

[36] V. Hudovernik *et al.*, "KumoRFM-2: Scaling Foundation Models for Relational Learning." 2026. Accessed: Nov. 05, 2026. [Online]. Available: arXiv:2604.12596v1

[37] NVIDIA Kumo, "Relational Deep Learning." Accessed: Nov. 05, 2026. [Online]. Available: NVIDIA Kumo documentation

[38] K. Jamieson and A. Talwalkar, "Non-stochastic Best Arm Identification and Hyperparameter Optimization," in *Proceedings of the 19th international conference on artificial intelligence and statistics*, 2016, pp. 240–248. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[39] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 18, no. 185, pp. 1–52, 2018, Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[40] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and Efficient Hyperparameter Optimization at Scale," in *Proceedings of the 35th international conference on machine learning*, 2018, pp. 1437–1446. Accessed: Oct. 26, 2026. [Online]. Available: Publisher

[41] M. Wang *et al.*, "4DBInfer: A 4D Benchmarking Toolbox for Graph-Centric Predictive Modeling on Relational DBs." 2024. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:2404.18209

[42] J. Motl and O. Schulte, "The CTU Prague Relational Learning Repository." 2015. Accessed: Oct. 26, 2026. [Online]. Available: arXiv:1511.03086v3